ClubSphere

University Club & Activity Lifecycle Management System

Combined Project Concept

A unified platform for discovering, managing, approving, participating in, and analyzing university clubs and campus activities.

Students	Club Managers	Faculty / HOD / Admin
Discover • Join • Register	Manage • Organize • Track	Approve • Monitor • Analyze

Design principle: treat the application as a club lifecycle and campus activity system, not merely as a club directory.

1. Project Idea

ClubSphere is a centralized university platform that brings fragmented club and event information into one system. Students can discover clubs and events, register and participate; club managers can operate their clubs and events; and university authorities can review club proposals, approve activities, monitor participation, and analyze performance.

The combined concept follows the lifecycle approach from the supplied reference while retaining the original problem: students currently have to search across scattered sources to find club information and opportunities.

2. Core Problem

- Club information is scattered across social media, messaging groups, posters, and informal contacts.
- Students do not have a single reliable place to discover active clubs and upcoming events.
- Club managers need a structured way to manage members, events, registrations, and outcomes.
- University authorities need a controlled workflow for creating, approving, and monitoring clubs.
- Event and club data exists, but is difficult to convert into useful participation and performance analytics.

3. Final Scope

Module	What it does
Club Discovery	Search, filter, and view university clubs, leadership, purpose, membership, achievements, and activity history.
Event Management	Publish events, define capacity/deadlines, collect registrations, track attendance, and record outcomes.
Club Lifecycle	Submit club proposals, route them through faculty/HOD approval, request revisions, approve/reject, and activate clubs.
Club Manager Dashboard	Manage club profile, members, events, registrations, attendance, outcomes, and performance.
Student Participation	Join/request membership, register for events, receive notifications, and view participation history.
Administration	Review proposals, monitor clubs/events, inspect analytics, and access audit trails.
Analytics	Measure club activity, registrations, attendance, participation, growth, and department-level trends.

4. Complete User Workflow

Student workflow

Login → discover clubs/events → view details → join a club or register for an event → receive confirmation → attend → participation is recorded → view history/certificates.

Club manager workflow

Login → manage club profile → create event → submit/publish event → receive registrations → manage capacity → record attendance → submit event outcome → view analytics.

New club workflow

Student/Applicant → submit club proposal → faculty coordinator review → HOD review → approved / rejected / changes requested → resubmission if required → club activation.

Administrative workflow

Authority login → review pending proposals/events → inspect evidence and history → take decision → system records reviewer, decision, timestamp, and comments → dashboards update.

5. DBMS — Core Data Layer

The DBMS stores operational university data and enforces relationships and integrity. The design should be normalized and support transactions, constraints, indexes, joins, aggregation, and reporting.

Entity	Purpose
Users	Students, managers, faculty, HODs, administrators, authentication and roles.
Departments	University departments and their association with users/clubs.
Clubs	Club profile, purpose, status, leadership, department, and metadata.
ClubMemberships	Many-to-many relationship between users and clubs.
Events	Event title, club, venue, date, capacity, deadline, status.
EventRegistrations	Students registered for events, with timestamps/status.
Attendance	Actual participation/attendance for events.
ClubProposals	New club applications and proposal information.
Approvals	Review decisions, reviewer, comments, timestamps, status.
EventOutcomes	Results, outcomes, metrics, reports, and evidence.
Notifications	In-app/email notification records.
AuditLogs	Important administrative/security actions and state changes.

Advanced DBMS direction: keep PostgreSQL/MySQL as the operational relational database. If time and infrastructure permit, selected historical/aggregated data can feed an analytical layer for long-term dashboards; this is an optional extension rather than a core dependency.

6. DSA — Algorithms with a Real Purpose

- Event search and filtering: efficient lookup over event/club attributes.
- Sorting: rank events by date, registration count, popularity, or relevance.
- Priority queue: prioritize high-demand or time-sensitive events for dashboards/notifications.
- Recommendation/ranking: score events using category interests, previous registrations, department relevance, popularity, and date.
- Optional graph model: represent relationships among students, clubs, departments, and interests for recommendation or community analysis.

The DSA layer should solve an application problem rather than exist only to demonstrate a textbook data structure.

7. System Design & Architecture

A modular monolith is recommended for a two-person student team: one deployable backend with clearly separated modules, rather than multiple independently deployed microservices.

```
React/Vite Frontend → REST API → Authentication/RBAC → Domain Modules → Relational DB → Analytics/Notifications
```

- Auth module — login, sessions/JWT, roles, permissions.
- Club module — profiles, membership, club lifecycle.
- Event module — creation, publishing, registration, attendance.
- Approval module — proposal/review workflow.
- Notification module — in-app/email events.
- Analytics module — aggregation and dashboard queries.
- Audit module — immutable-style records of important actions.

8. Cloud & Deployment Architecture

The cloud architecture should demonstrate deployment and managed services without overengineering the project.

```
Frontend hosting → Backend/API hosting → Managed PostgreSQL → Object storage for posters/proposals → Email/notification provider → Monitoring/logging
```

Optional advanced architecture: a cache such as Redis can be introduced for high-read data such as upcoming events; object storage can hold proposal documents and event media; an analytics warehouse/data pipeline can be added if the project requires a deeper cloud-data-engineering demonstration.

9. Cloud Security & Governance

Security area	Implementation
Authentication	Secure login, password hashing, session/JWT management.
RBAC	Student, Club Manager, Faculty, HOD, Admin roles with least-privilege permissions.
Authorization	Backend checks permissions for every protected operation.
Secrets	API keys/database credentials stored in environment/secret management, not source code.
Transport	HTTPS/TLS in deployment.
Input security	Schema validation, sanitization, safe file upload validation.
API protection	Rate limiting and sensible request validation.
Auditability	Record sensitive approvals, changes, and administrative actions.

Security area	Implementation
Data protection	Restrict access to student/contact data and avoid exposing unnecessary personal information.

10. Integration

- Frontend ↔ Backend: REST APIs for clubs, events, registrations, proposals, approvals, and analytics.
- Backend ↔ Database: transactional reads/writes with validation and constraints.
- Cloud storage: event posters and proposal documents.
- Notifications: email and/or in-app notifications for registrations and approvals.
- Optional QR attendance: event-specific QR codes can simplify attendance recording.
- Optional calendar integration: approved events can be exposed to a calendar workflow.

11. Analytics & Dashboards

- Student: clubs joined, events registered/attended, participation history.
- Club manager: member count, events conducted, registrations, attendance rate, growth, outcomes.
- HOD/Admin: active clubs, pending proposals, department participation, event performance, most/least active clubs.
- System: registration trends, attendance rate, capacity utilization, club growth, event success indicators.

Example club dashboard

Members: 87 | Events: 12 | Registrations: 643 | Average attendance: 72% | Projects/outcomes: 8

12. Audit & Lifecycle Tracking

Important actions should be traceable. Example: “HOD approved Club #42 at 10:32”, “Manager created Event #91 at 11:05”, or “Faculty requested changes to Proposal #1042”. This supports accountability, troubleshooting, and security review.

13. Recommended Technology Stack

Layer	Recommended choice
Frontend	React + Vite + Tailwind CSS
Backend	Python + FastAPI
Database	PostgreSQL
Authentication	JWT or secure session-based authentication
Object storage	S3-compatible/Azure Blob storage, optional depending on hosting choice
Caching	Redis, optional
Analytics	SQL-based dashboards initially; warehouse/pipeline as an advanced extension

Layer	Recommended choice
Deployment	Any suitable cloud platform with managed database support
Version control	Git + GitHub

14. Requirement-to-Implementation Mapping

Project area	What ClubSphere demonstrates
System Design	Modular architecture, workflows, APIs, roles, data flow, scalability decisions.
DBMS	Relational schema, normalization, constraints, joins, transactions, indexes, analytics queries.
DSA	Search, sorting, priority queues, ranking/recommendation algorithms.
Cloud Security	RBAC, authentication, authorization, HTTPS, secrets, secure uploads, audit logs.
Integration	Frontend/backend/database integration plus storage, notifications, and optional QR/calendar integrations.
Cloud/Data Architecture	Managed database, object storage, deployment, optional cache and analytical pipeline.
Final Demo	Student portal + club manager dashboard + HOD/admin dashboard + end-to-end approval/event lifecycle.

15. Recommended Build Strategy

Core MVP — build these first:

- Authentication + roles
- Club directory and club profiles
- Event creation and discovery
- Event registration and capacity handling
- Club proposal + faculty/HOD approval workflow
- Club manager dashboard
- HOD/admin dashboard
- Attendance and basic analytics
- Audit logs

Advanced extensions — only after the core system works:

- Recommendation/ranking engine
- Redis caching
- QR-based attendance
- Email/calendar integrations

- Object storage for documents/media
- Analytical warehouse + ETL/data pipeline
- Advanced anomaly/participation insights

16. Final One-Line Description

ClubSphere is a secure, role-based university club and activity lifecycle platform that centralizes club discovery, event participation, club creation/approval, activity management, attendance, analytics, and institutional governance.

Source basis: the supplied Integrated Full-Stack Project Student Guide and the supplied Gemini discussion/reference. The combined design intentionally retains the original club/event problem while incorporating the stronger lifecycle, DBMS, cloud, analytics, security, and architecture elements discussed in the reference.